

SENA - Servicio Nacional de Aprendizaje

Centro de Formación - Tecnología en Desarrollo de Software

Manual de Estándares de Codificación del Proyecto

ejemplo01

Ficha SENA: 3118526

Autor: Jackson Darley Montoya Mercado

Tecnologías: Angular 22 / TypeScript / Angular Material / Tailwind CSS

Fecha: 03/08/2026

1. Introducción

El presente manual establece las normas, convenciones y buenas prácticas de codificación que deben seguirse en el desarrollo del proyecto **ejemplo01**. El objetivo es garantizar la calidad, la legibilidad, la mantenibilidad y la uniformidad del código fuente, facilitando el trabajo colaborativo entre los miembros del equipo y la evolución futura del sistema.

Este documento se encuentra alojado en la carpeta **docs/** del repositorio y debe ser consultado por todo desarrollador que realice cambios en el código.

1.1 Alcance

El manual aplica a la totalidad del código fuente del proyecto, incluyendo componentes, servicios, modelos, guardas, rutas, configuración y estilos. La aplicación es un **sistema de gestión empresarial** que administra usuarios, productos, clientes y facturas, construido sobre Angular 22 con TypeScript, Angular Material y Tailwind CSS.

1.2 Público objetivo

Desarrolladores frontend, desarrolladores fullstack, revisores de código y administradores del repositorio que participen en el ciclo de vida del proyecto.

2. Convenciones de Nomenclatura

Se adoptan las siguientes convenciones de nomenclatura, adaptadas a la naturaleza del proyecto (TypeScript / Angular).

Elemento	Convención	Ejemplo
Clases e Interfaces	PascalCase	Usuario, AuthResponse, Credenciales
Componentes Angular	PascalCase + sufijo .component	LoginComponent, UsuariosComponent, LayoutComponent
Servicios Angular	PascalCase + sufijo .service	AuthService, UsuariosService, FacturasService
Modelos / Interfaces	PascalCase + sufijo .model	Usuario, Producto, Cliente, Factura
Métodos y Variables	camelCase	obtenerUsuarios(), fechaCreacion, esActivo
Guardas	PascalCase + sufijo .guard	AuthGuard
Constantes	UPPER_SNAKE_CASE	API_URL, MAX_INTENTOS
Base de datos (campos)	snake_case	fecha_creacion, numero_documento
Archivos y carpetas	kebab-case	login.component.ts, usuarios.service.ts

3. Estructura de Directorios

El proyecto sigue la estructura recomendada por Angular para una aplicación modular y escalable. La carpeta **docs/** aloja la documentación técnica.

```
ejemplo01/ |-- .angular/ # Configuración y cache de Angular CLI |-- .gitignore # Archivos
ignorados por Git |-- .vscode/ # Configuración del editor |-- angular.json # Configuración
del workspace Angular |-- docs/ # Documentación del proyecto | `--
Estandares_Codificación.pdf # Este manual |-- public/ # Archivos estáticos públicos |-- src/
| |-- app/ | | |-- app.config.ts # Configuración de la aplicación | | |-- app.routes.ts #
Definición de rutas | | |-- app.ts # Componente raíz | | |-- components/ | | | |-- login/ #
Componente de inicio de sesión | | | |-- dashboard/ # Panel principal | | | |-- usuarios/ #
Gestión de usuarios | | | |-- productos/ # Gestión de productos | | | |-- clientes/ # Gestión
de clientes | | | |-- facturas/ # Gestión de facturas | | | `-- layout/ # Layout general de
la app | | |-- guards/ # Guardas de autenticación | | | |-- auth.guard.ts # Protección de
rutas | | |-- models/ # Modelos de datos | | | |-- usuario.model.ts | | | |--
producto.model.ts | | | |-- cliente.model.ts | | | `-- factura.model.ts | | `-- services/ #
Servicios de negocio | | |-- auth.service.ts | | |-- usuarios.service.ts | | |--
productos.service.ts | | |-- clientes.service.ts | | `-- facturas.service.ts | |--
index.html # Página principal | |-- main.ts # Punto de entrada | `-- styles.css # Estilos
globales (Tailwind) |-- package.json # Dependencias y scripts |-- tsconfig.app.json #
Configuración TS de la app |-- tsconfig.json # Configuración TS base `-- tsconfig.spec.json
# Configuración TS de pruebas
```

4. Buenas Prácticas de Control de Versiones (Git)

El proyecto se gestiona con Git y se aloja en GitHub. Se siguen las siguientes buenas prácticas para mantener un historial limpio y trazable.

4.1 Convención de commits

Cada commit debe tener un mensaje descriptivo que comience con un prefijo estándar que indique el tipo de cambio realizado. Formato recomendado:

[tipo]: [descripción resumida en presente]

Prefijo	Uso	Ejemplo
feat:	Nueva funcionalidad	feat: agregar módulo de facturación
fix:	Corrección de errores	fix: corregir validación de login
docs:	Cambios en documentación	docs: actualizar manual de usuario
refactor:	Mejora de código sin cambiar comportamiento	refactor: simplificar servicio de usuarios
style:	Cambios de formato sin afectar lógica	style: aplicar formato prettier
test:	Añadir o modificar pruebas	test: agregar pruebas de auth
chore:	Tareas de mantenimiento	chore: actualizar dependencias

4.2 Flujo de trabajo recomendado

1. Crear una rama descriptiva por cada funcionalidad (ej. feature/login).
2. Realizar commits pequeños y atómicos con mensajes claros.
3. Antes de cada commit, revisar los cambios con `git status` y `git diff`.
4. Mantener el código formateado (Prettier) y sin errores de lint.
5. Integrar los cambios a la rama principal (main) mediante pull request.

4.3 Ejemplo de flujo de trabajo

1. Verificar estado del repositorio `git status` # 2. Añadir los archivos modificados `git add docs/Estandares_Codificacion.pdf` # 3. Crear un commit con mensaje descriptivo `git commit -m "docs: agregar manual de estandares de codificacion en carpeta docs"` # 4. Subir los cambios a la rama principal `git push origin main`

5. Herramientas y Calidad de Código

Herramienta	Propósito
TypeScript	Lenguaje principal con tipado estático.
Angular CLI	Generación de componentes, servicios y builds.
Angular Material	Componentes de interfaz reutilizables.
Tailwind CSS	Estilos utilitarios y diseño responsive.
Prettier	Formateo automático del código.
Vitest	Pruebas unitarias de componentes y servicios.
Git / GitHub	Control de versiones y alojamiento del repositorio.

Este manual fue generado de forma automatizada mediante un script de Python (reportlab) y se encuentra versionado en la carpeta **docs/** del repositorio **ejemplo01**.